Linked List

Insert and Delete

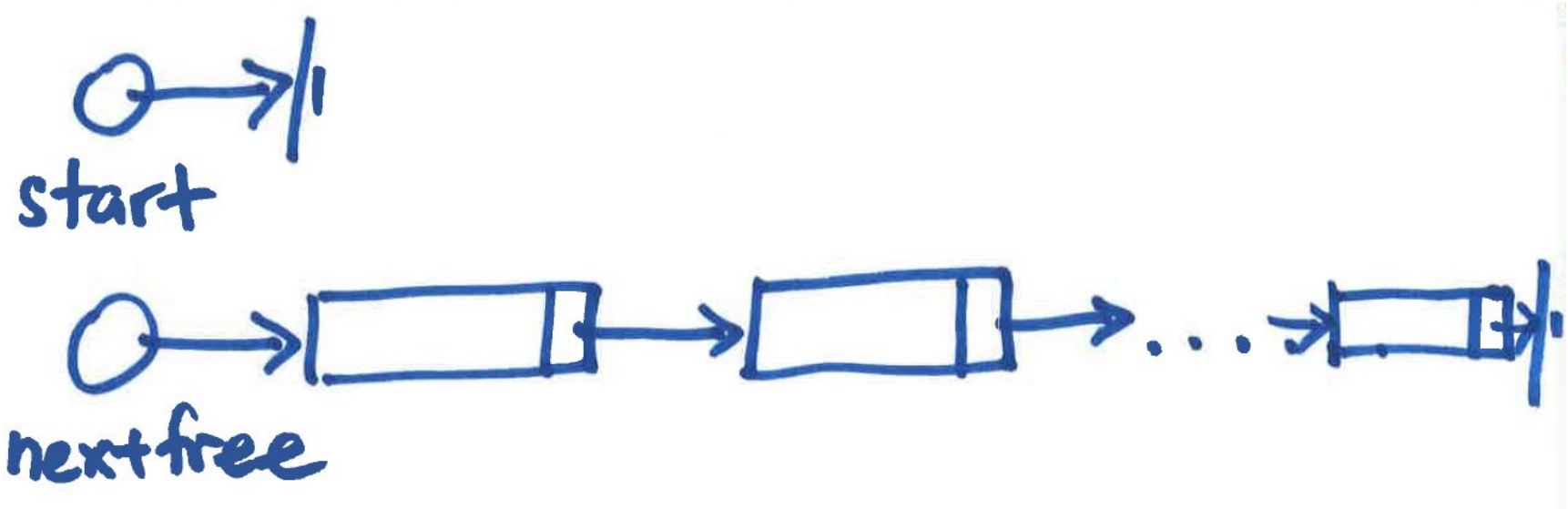
Insert into Linked List

```
PROCEDURE Insert(newData)
    IF nextfree IS NULL THEN                          //check if free node exists
        output("No free node available") and RETURN
    ENDIF
    Make[nextfree].name ← newData                     //store newData in free node
    IF start IS NULL THEN                             //store into empty linked list
        temp ← Make[nextfree].pointer
        Make[nextfree].pointer ← NULL
        start ← nextfree
        nextfree ← temp
    ELSE
        previous ← NULL
        current ← start
        WHILE current IS NOT NULL                     //traverse from start of linked list
            IF newData < Make[current].name THEN
                BREAK                 //Position found between previous and current node
            ENDIF
            previous ← current
            current ← Make[current].pointer
        ENDWHILE
        IF previous IS NULL THEN                              // store as first node
            temp ← Make[nextfree].pointer
            Make[nextfree].pointer ← start
            start ← nextfree
            nextfree ← temp
        ELSE                                      // store between previous and current
            temp ← Make[nextfree].pointer
            Make[nextfree].pointer ← current
            Make[previous].pointer ← nextfree
            nextfree ← temp
        ENDIF
    ENDIF
ENDPROCEDURE
```

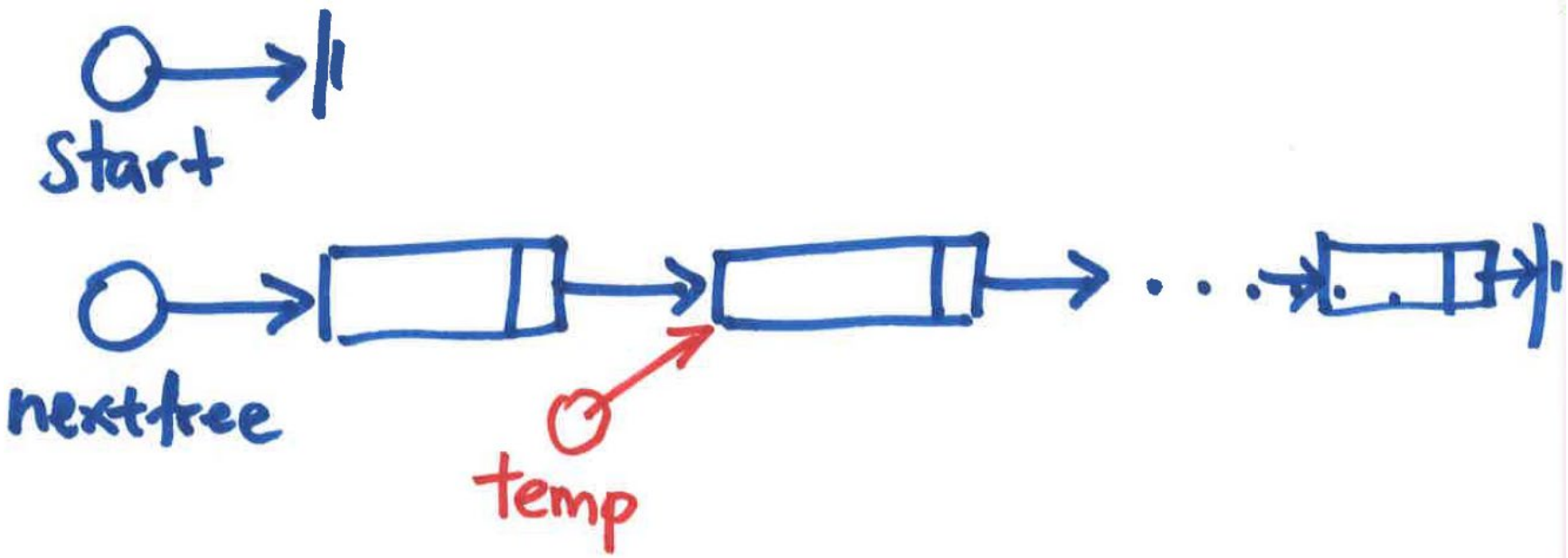
```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```

Insert into empty linked list

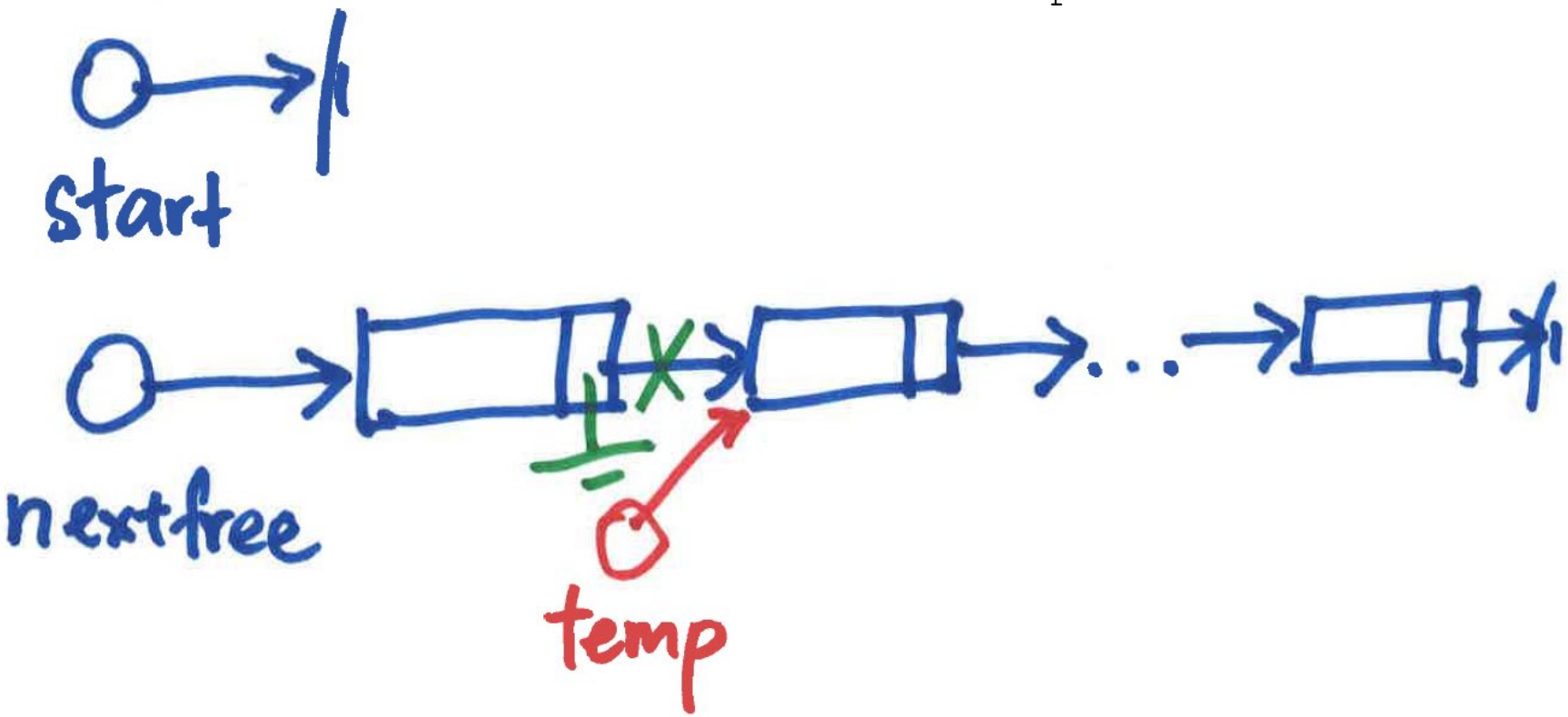
```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```



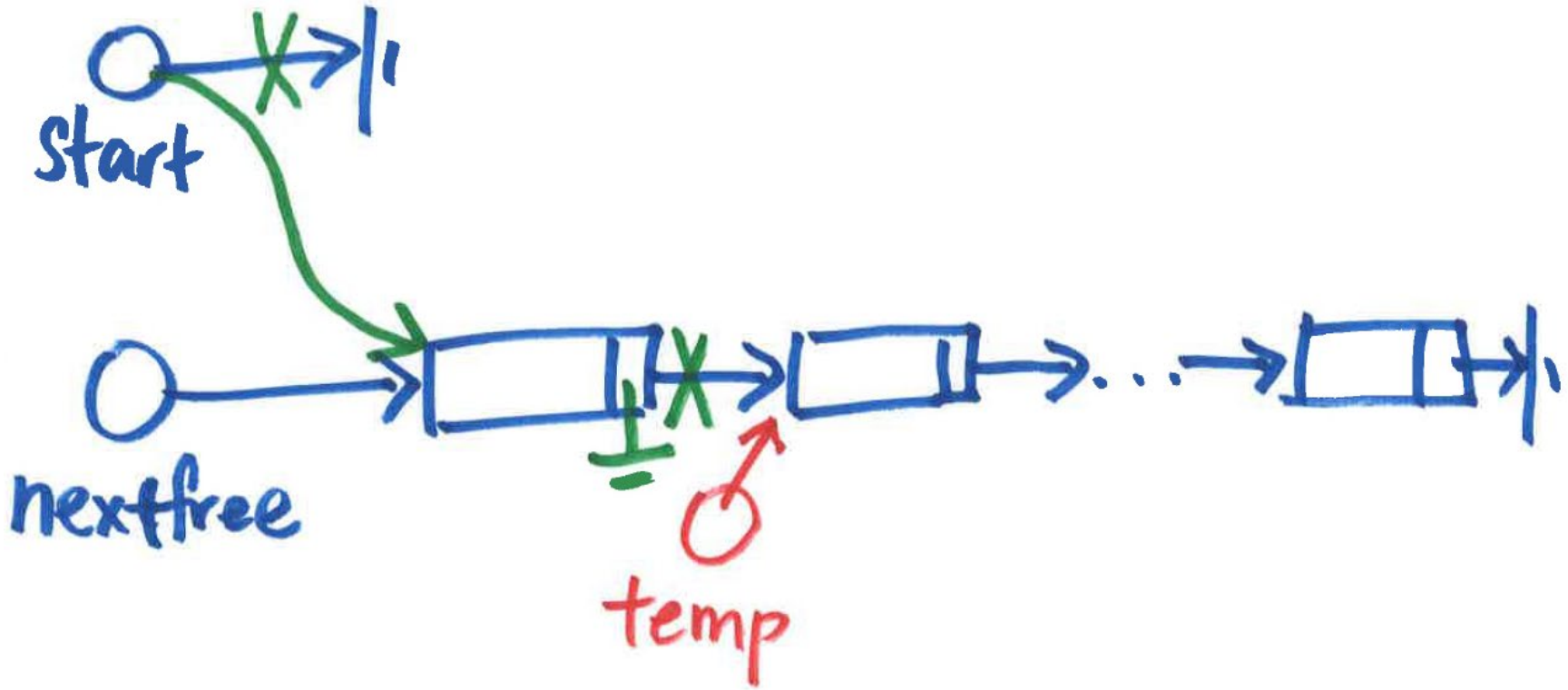
```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```



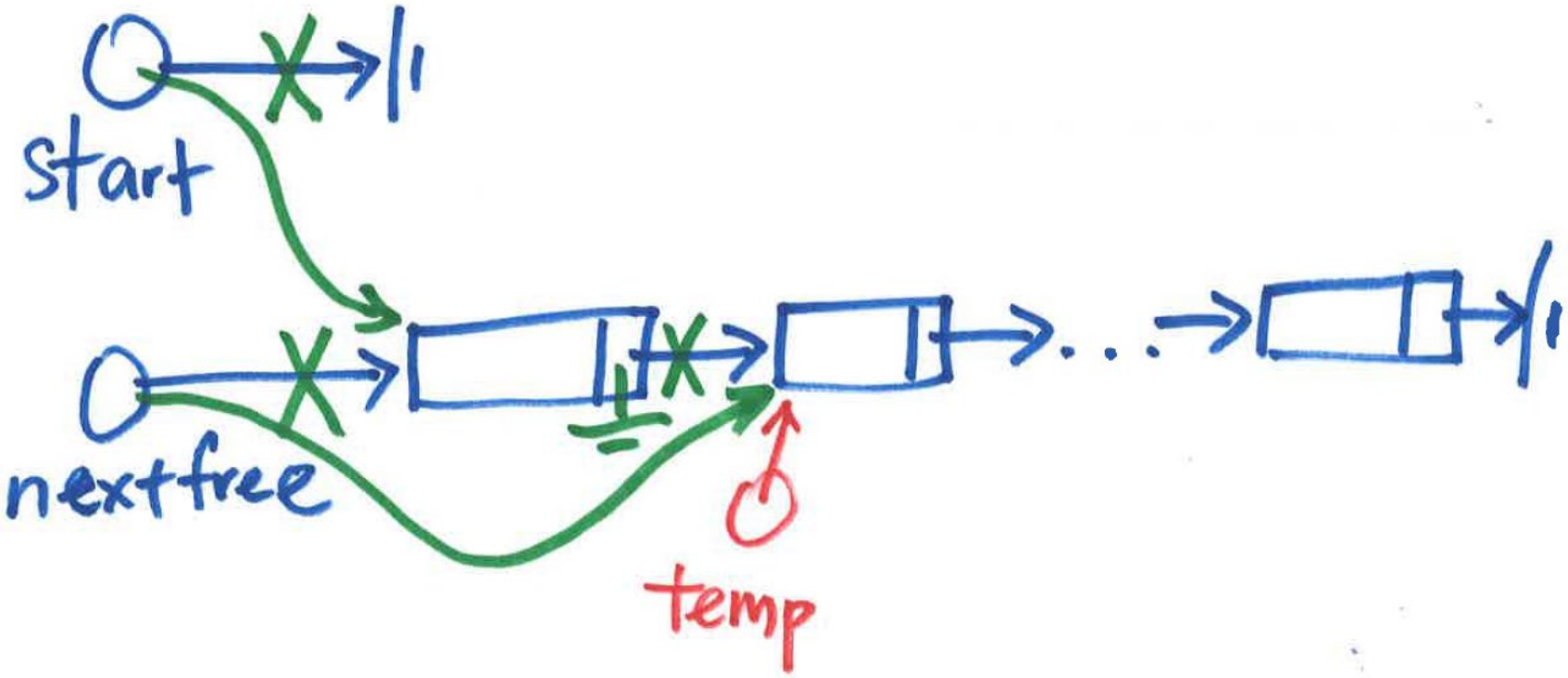
```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```



```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```



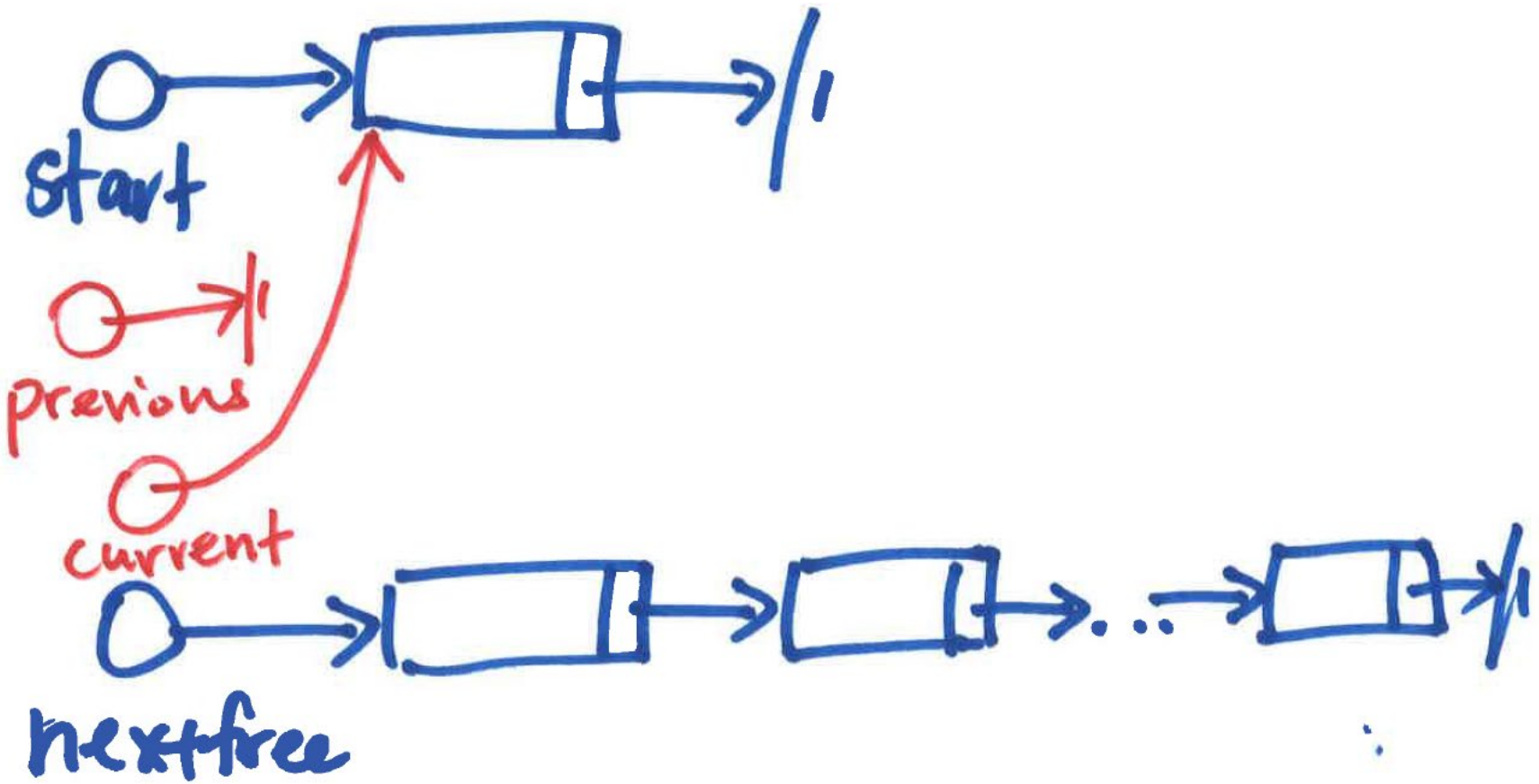
```
IF start IS NULL THEN                    //store into empty linked list
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← NULL
    start ← nextfree
    nextfree ← temp
```



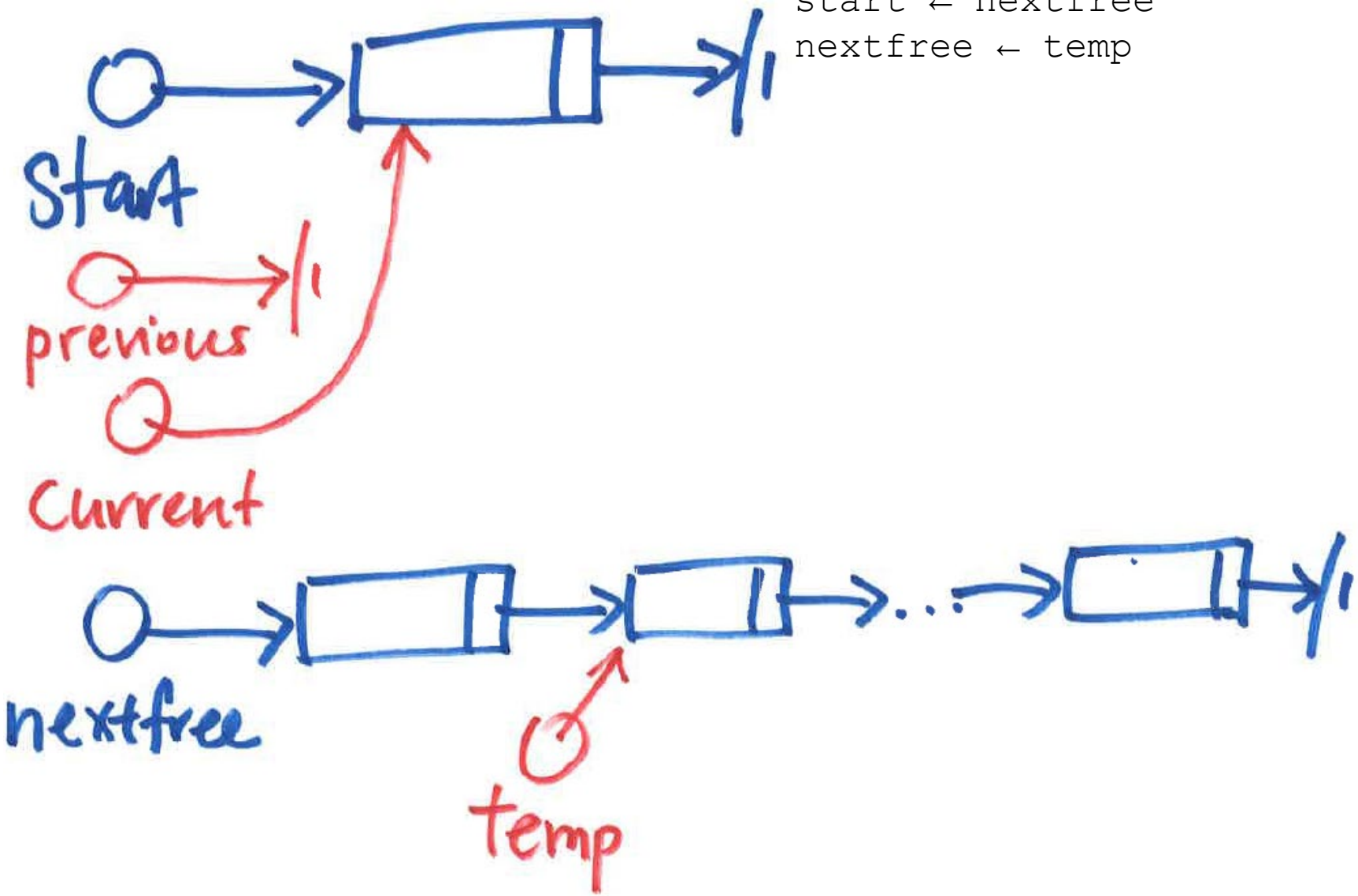
```
IF previous IS NULL THEN      // store as first node
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← start
    start ← nextfree
    nextfree ← temp
```

Insert as 1st node

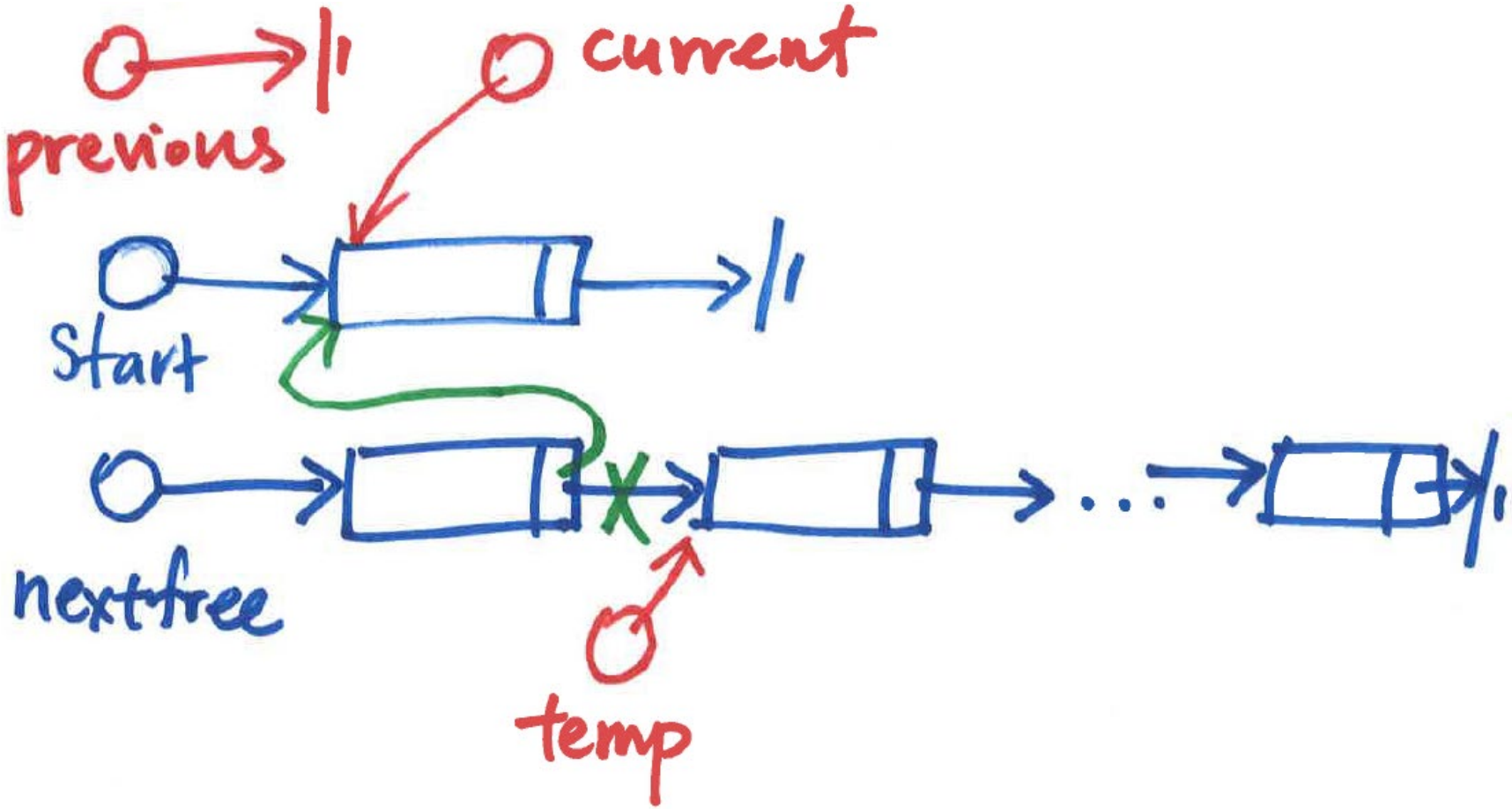
```
previous ← NULL
current ← start
```



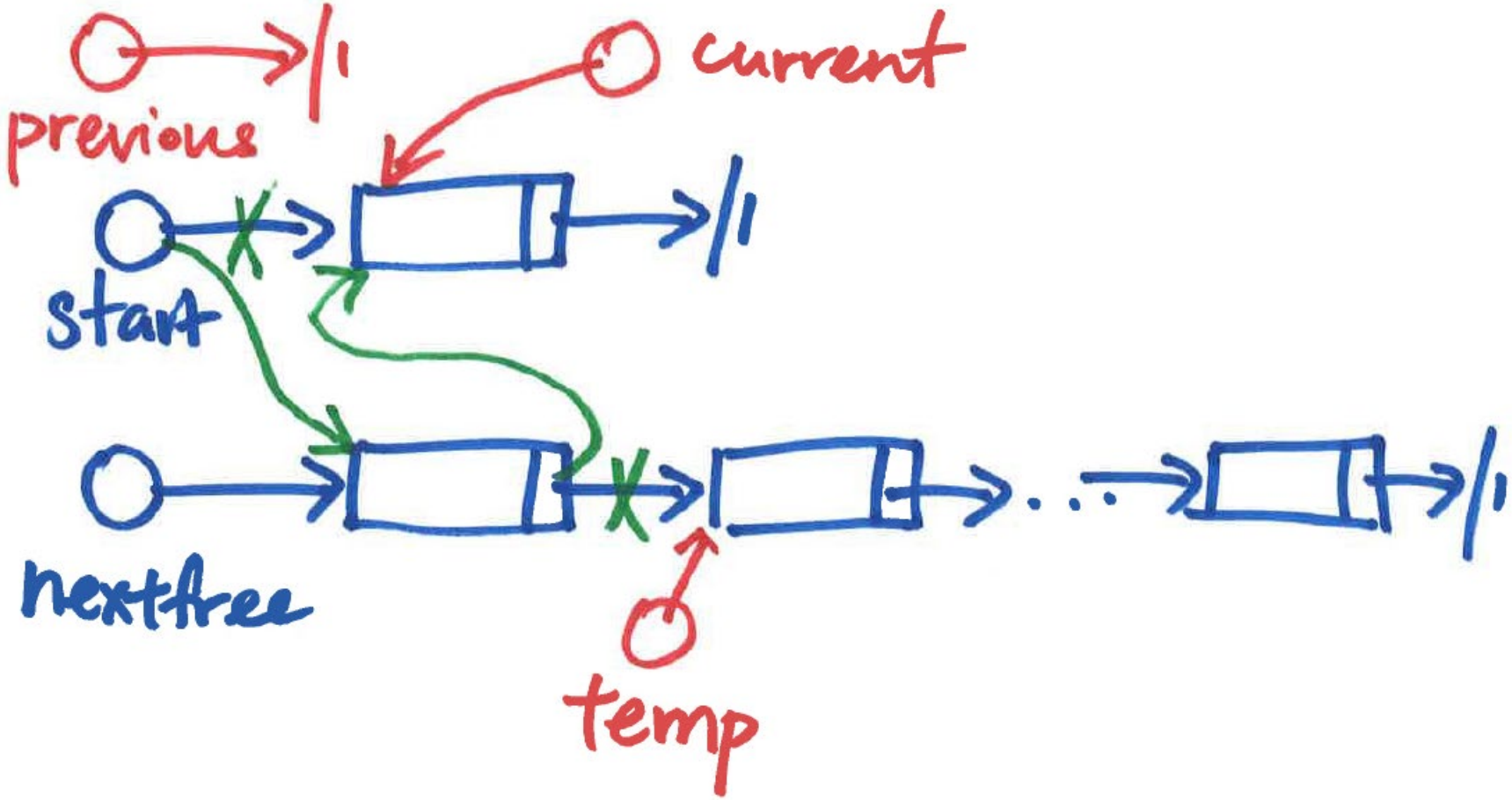
```
IF previous IS NULL THEN      // store as first node
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← start
    start ← nextfree
    nextfree ← temp
```



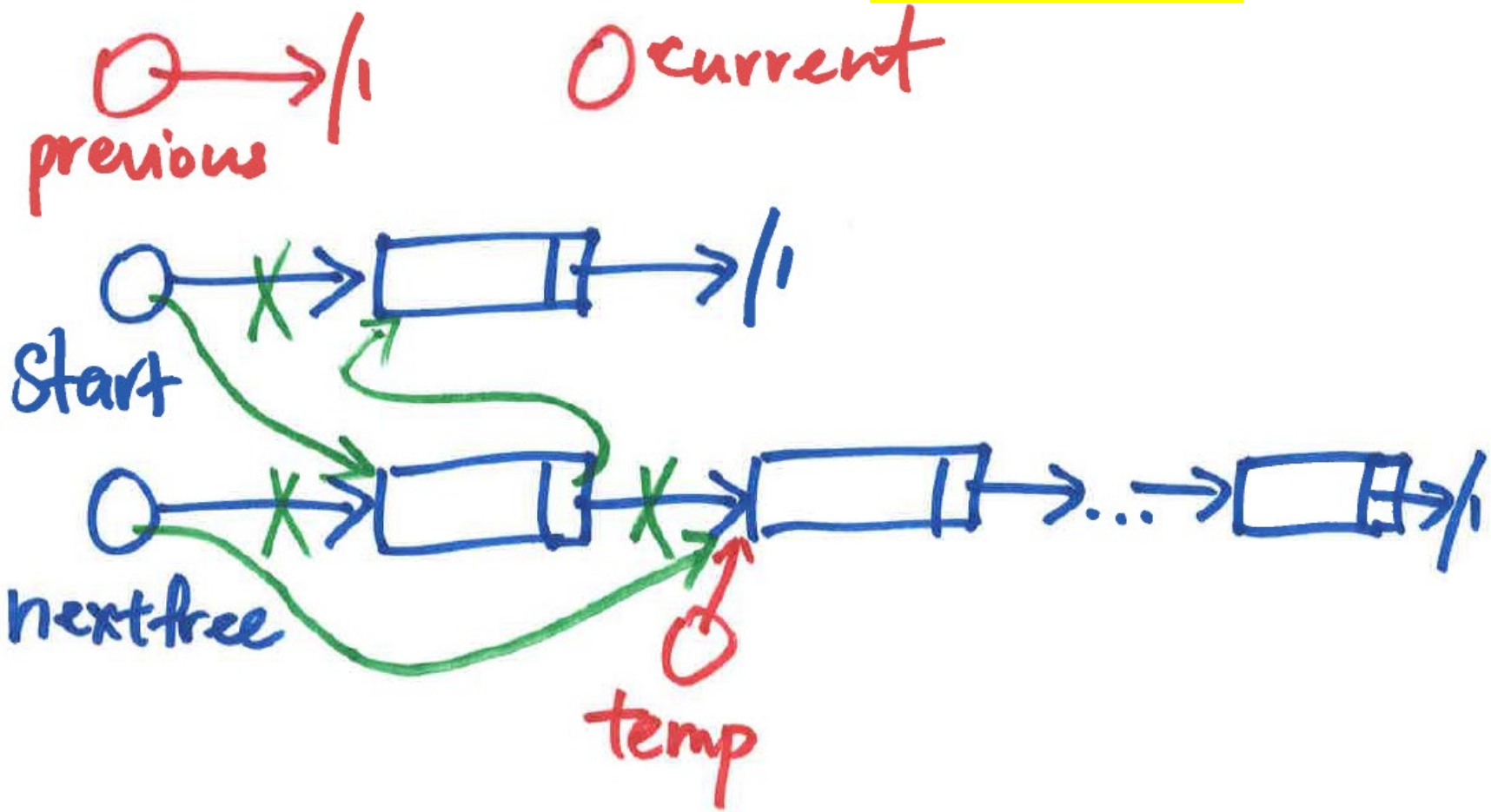
```
IF previous IS NULL THEN      // store as first node
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← start
    start ← nextfree
    nextfree ← temp
```



```
IF previous IS NULL THEN      // store as first node
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← start
    start ← nextfree
    nextfree ← temp
```



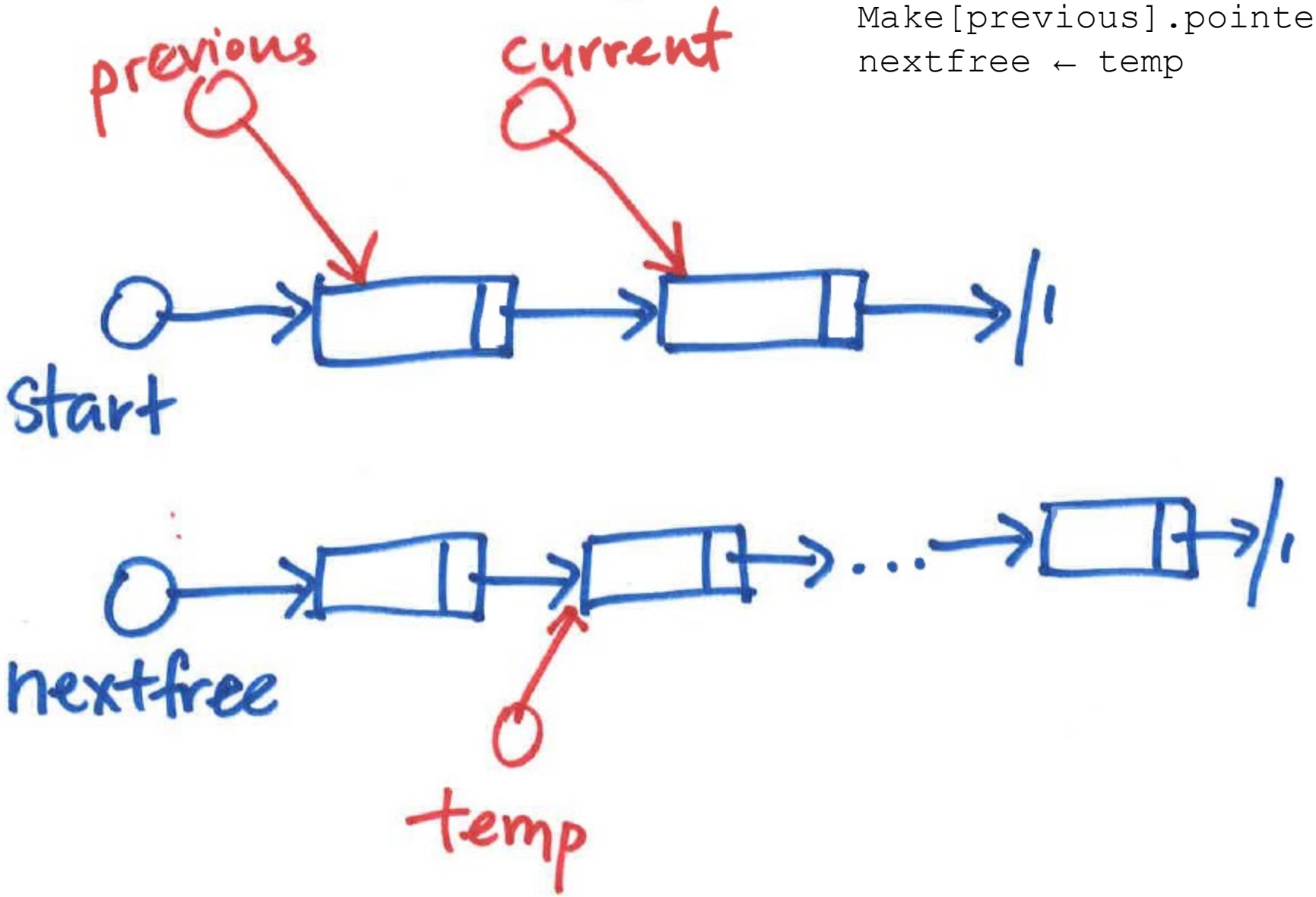
```
IF previous IS NULL THEN      // store as first node
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← start
    start ← nextfree
    nextfree ← temp
```



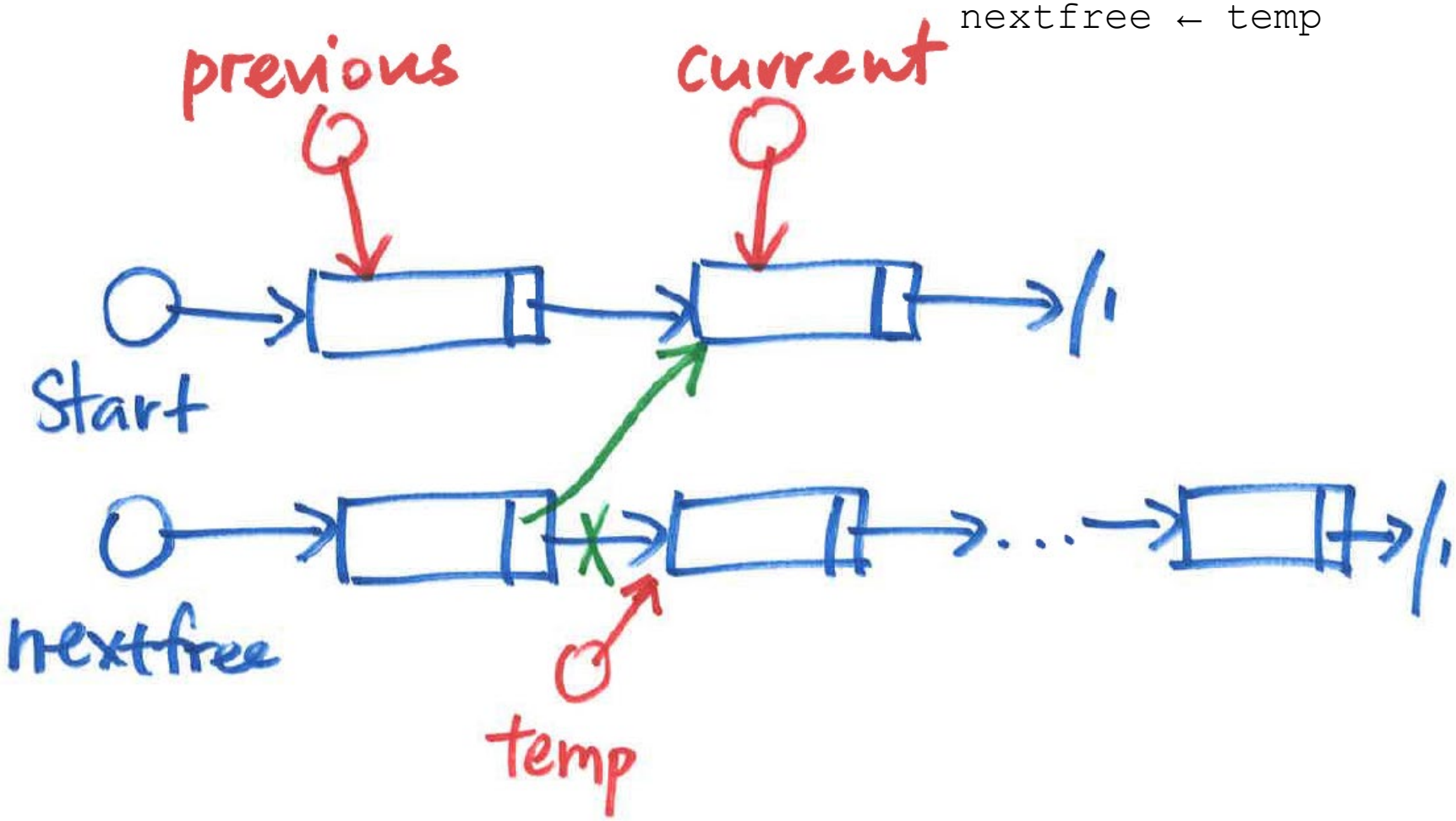
```
ELSE      // store between previous and current
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← current
    Make[previous].pointer ← nextfree
    nextfree ← temp
```

Insert as non-1st node

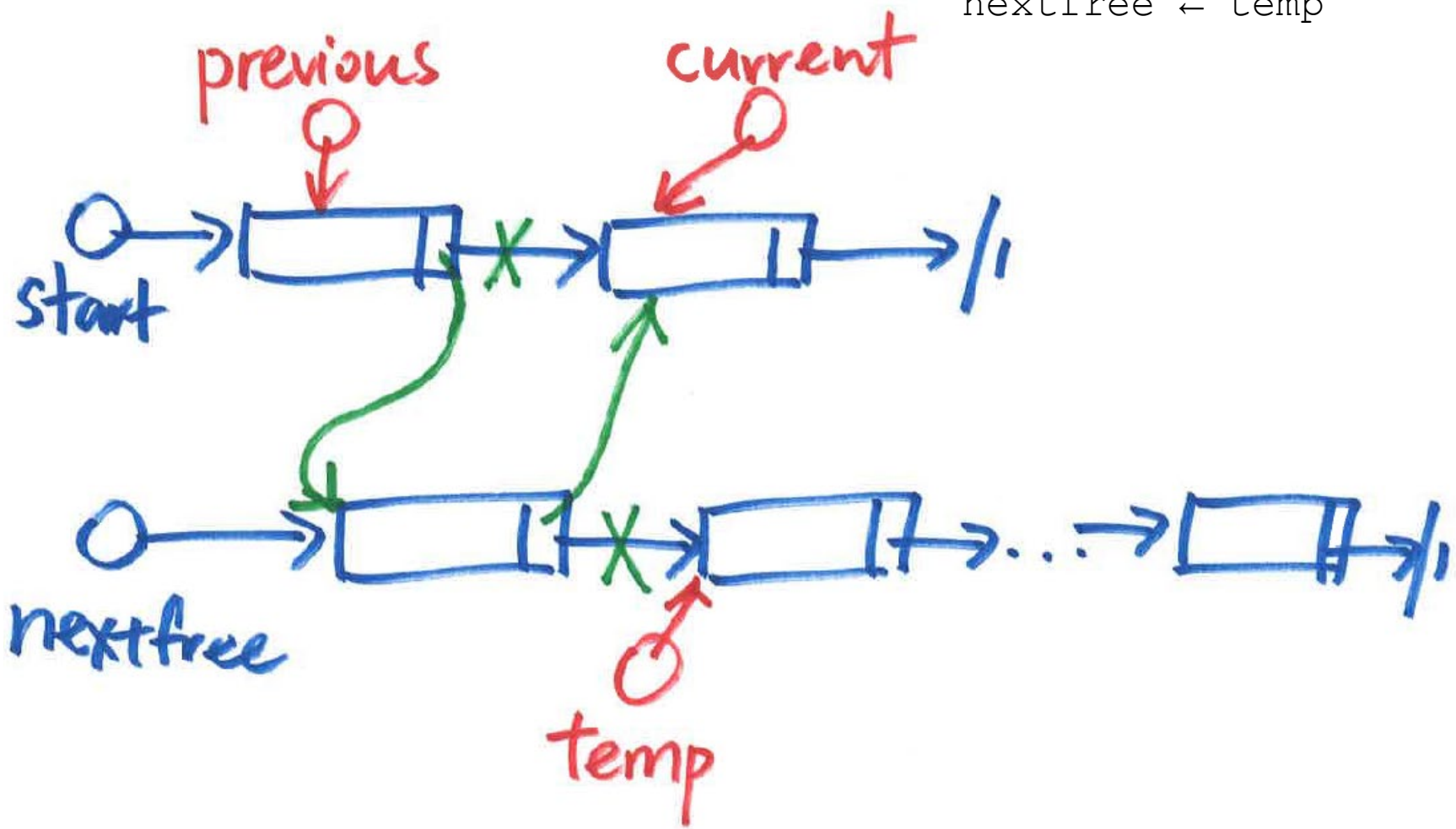
```
ELSE        // store between previous and current
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← current
    Make[previous].pointer ← nextfree
    nextfree ← temp
```



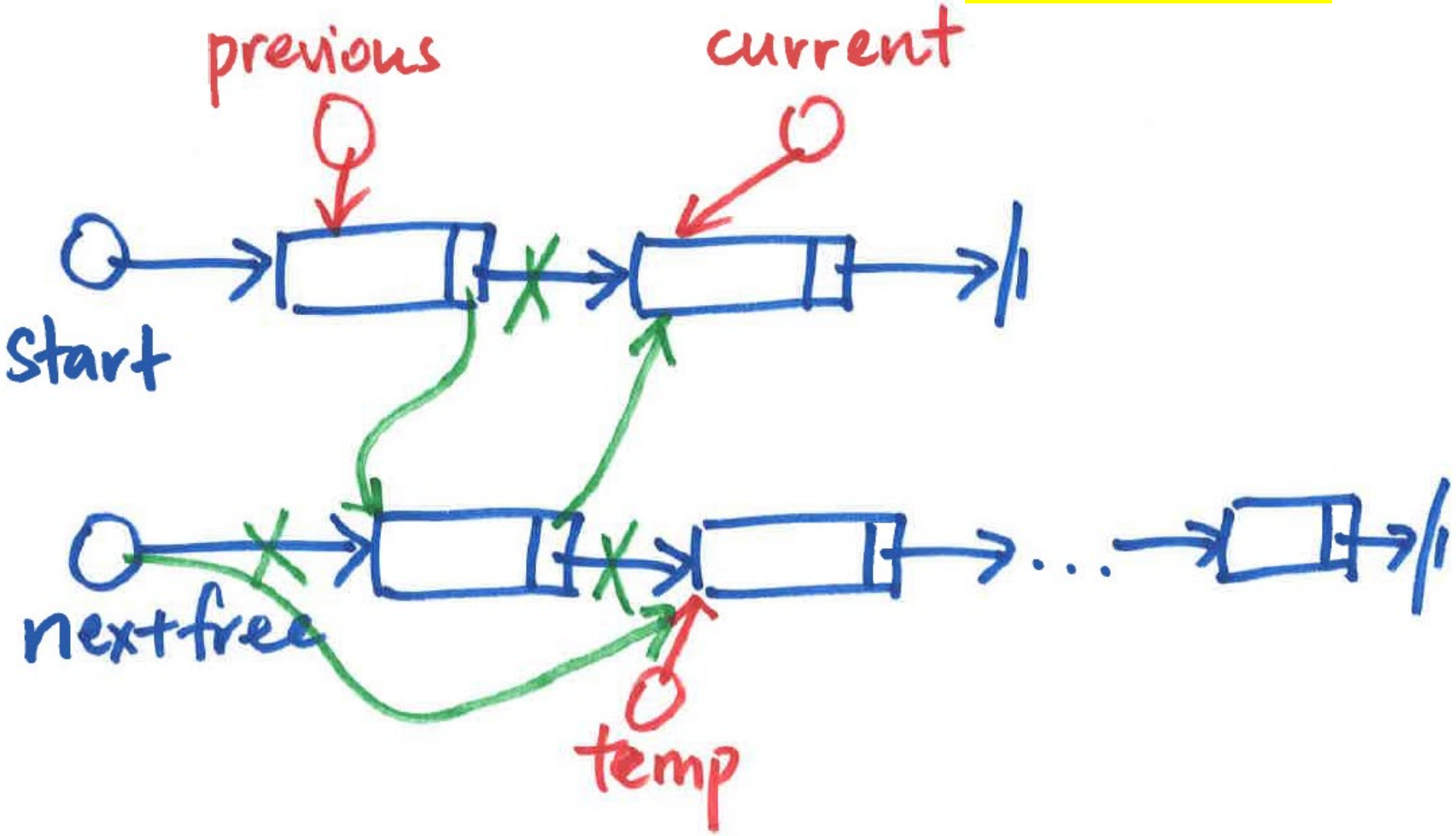
```
ELSE        // store between previous and current
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← current
    Make[previous].pointer ← nextfree
    nextfree ← temp
```



```
ELSE        // store between previous and current
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← current
    Make[previous].pointer ← nextfree
    nextfree ← temp
```



```
ELSE        // store between previous and current
    temp ← Make[nextfree].pointer
    Make[nextfree].pointer ← current
    Make[previous].pointer ← nextfree
    nextfree ← temp
```



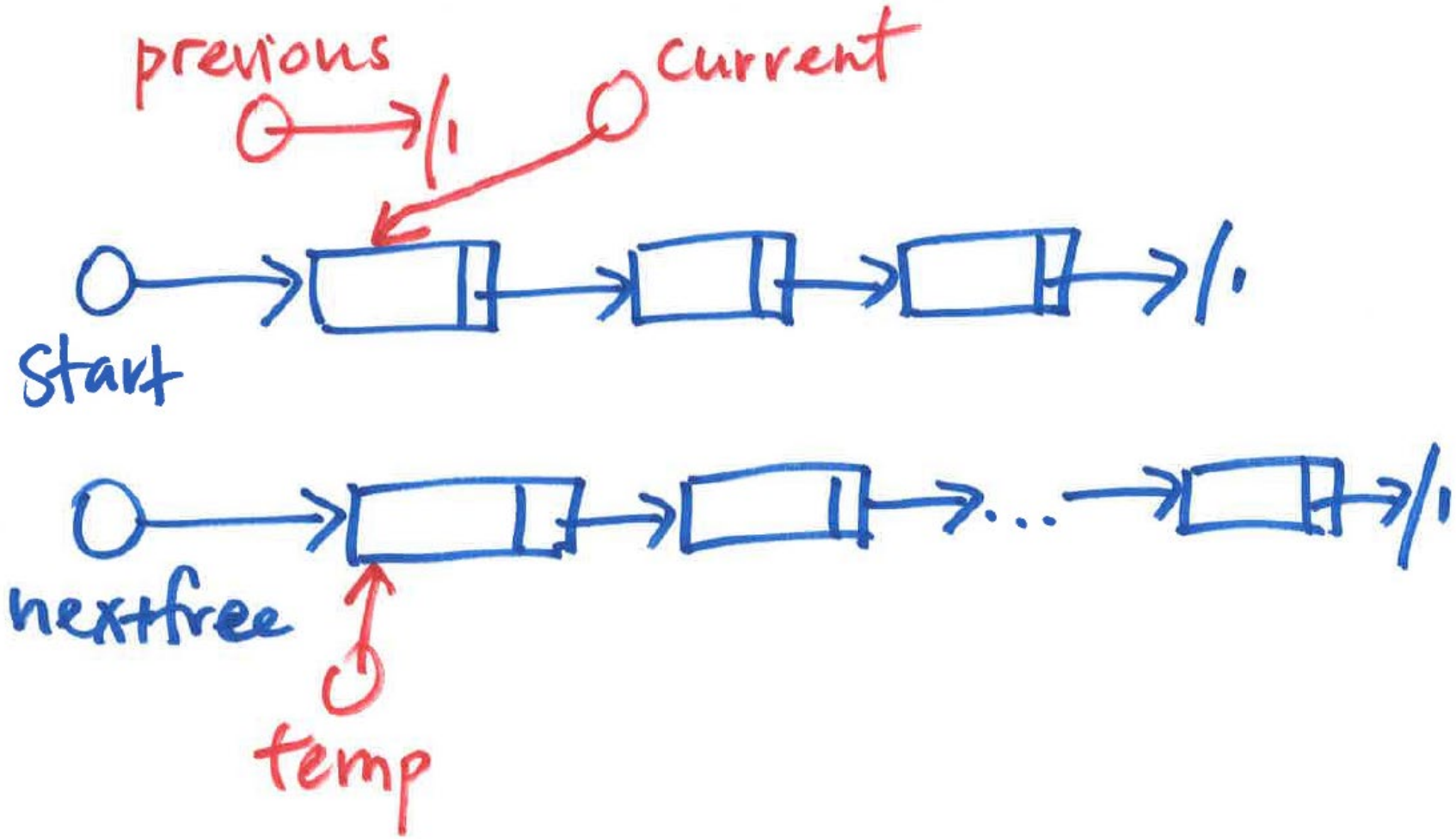
Delete from Linked List

```
PROCEDURE Delete(Data)
    IF start IS NULL THEN                          //check if linked list is empty
        output("List is empty!") and RETURN
    ENDIF
    previous ← NULL
    current ← start
    WHILE current IS NOT NULL                      //traverse from start of linked list
        IF Data = Make[current].name THEN          //Data to be deleted is found
            read data and BREAK                    //current points to node to be deleted
        ENDIF
        previous ← current
        current ← Make[current].pointer
    ENDWHILE
    IF current IS NULL THEN                        //reach end of linked list, node not found
        output("Node not found") and RETURN
    ENDIF
    IF previous IS NULL                            //first node to be deleted
        temp ← nextfree
        nextfree ← current
        start ← Make[current].pointer
        Make[current].pointer ← temp
    ELSE                                           //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
    ENDIF
ENDPROCEDURE
```

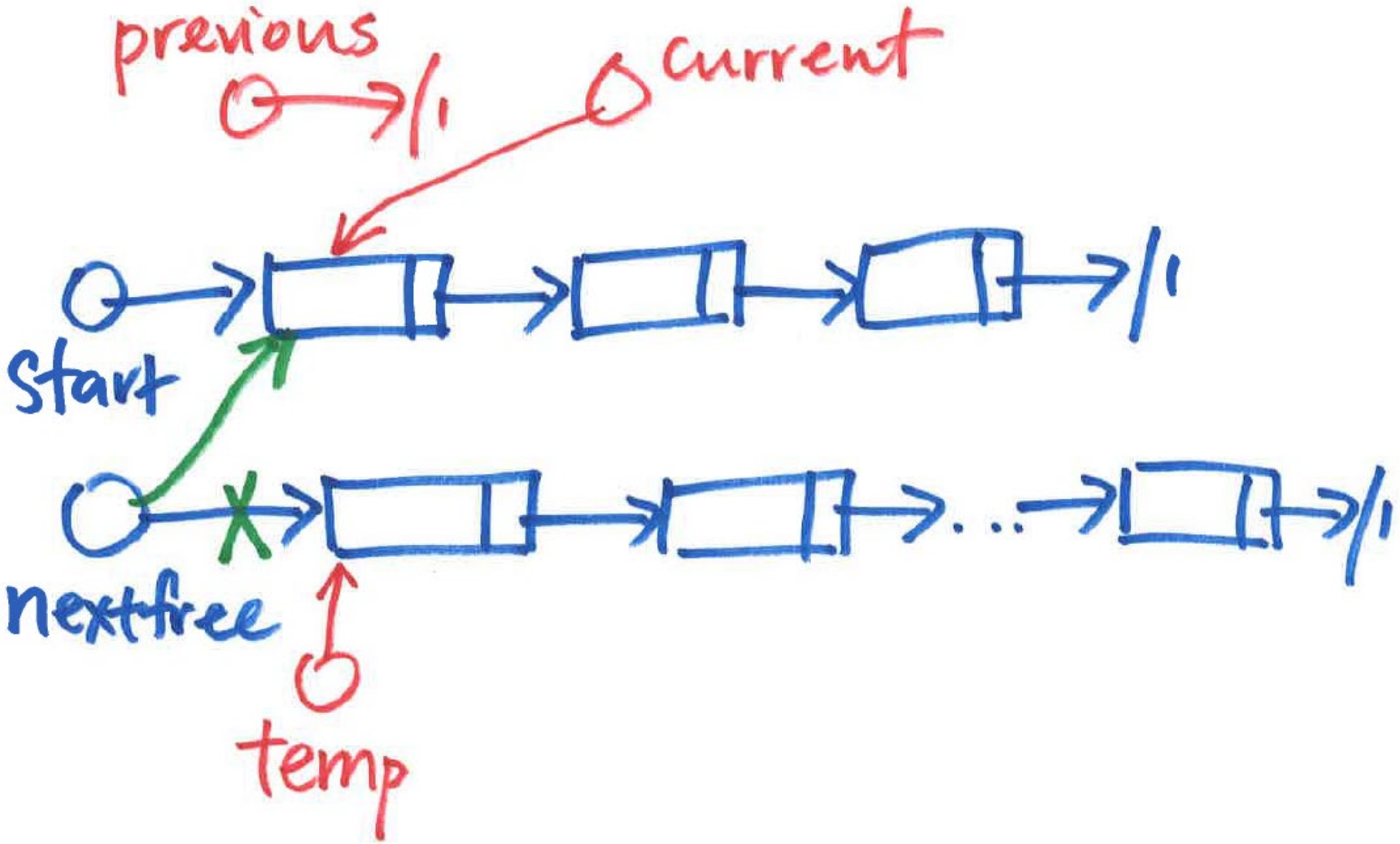
```
IF previous IS NULL   //first node to be deleted
    temp ← nextfree
    nextfree ← current
    start ← Make[current].pointer
    Make[current].pointer ← temp
```

Delete 1st node

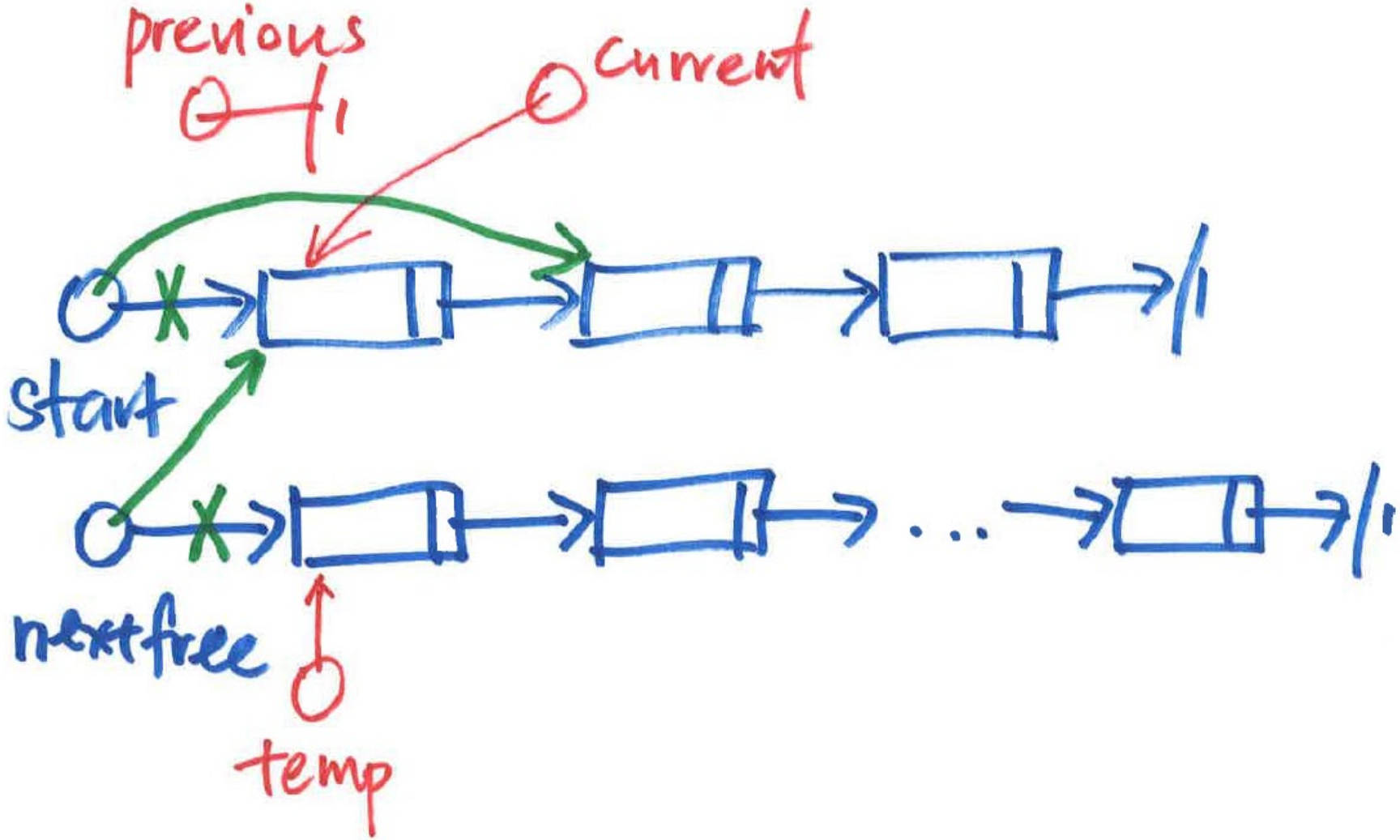
```
IF previous IS NULL   //first node to be deleted
    temp ← nextfree
    nextfree ← current
    start ← Make[current].pointer
    Make[current].pointer ← temp
```



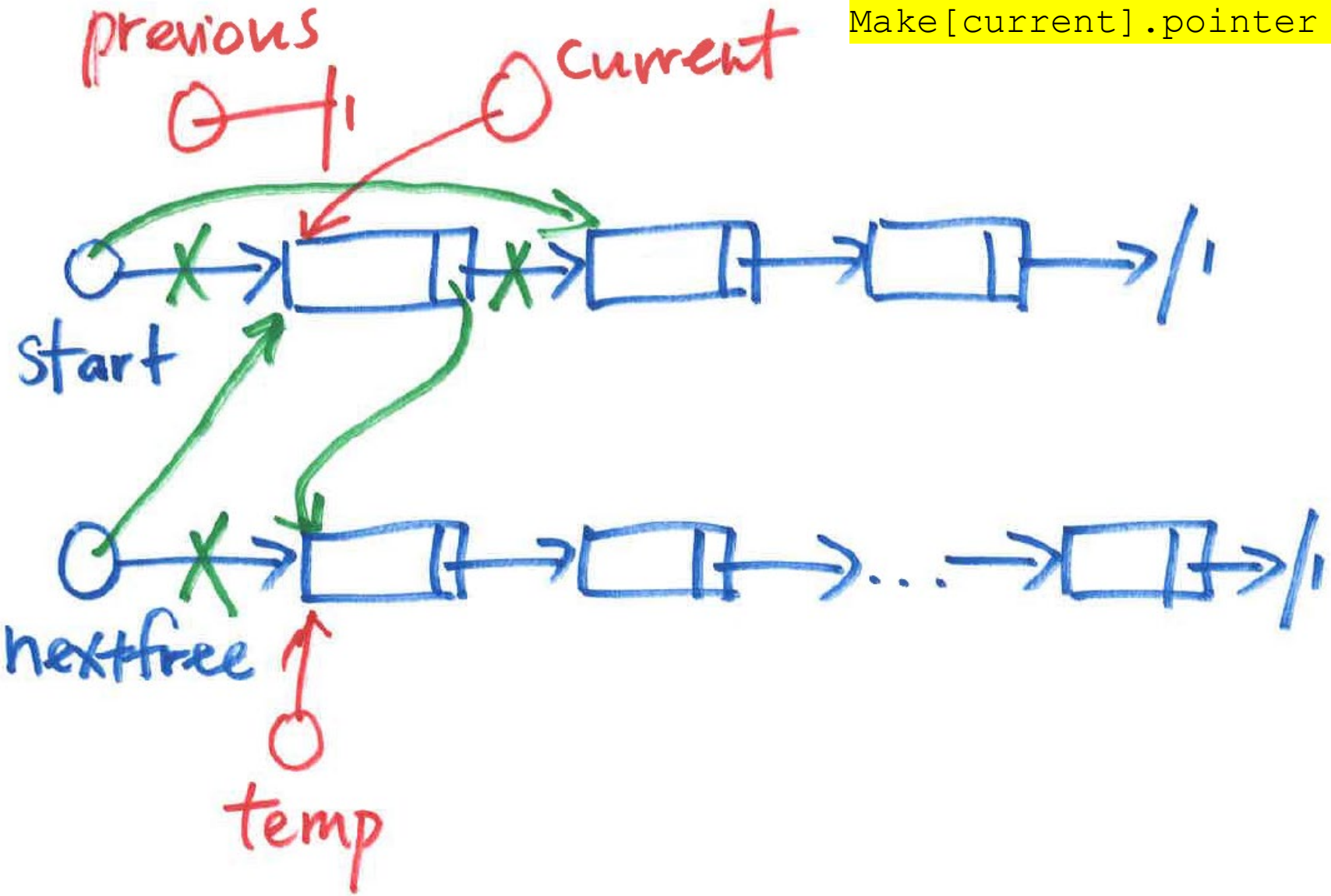
```
IF previous IS NULL   //first node to be deleted
    temp ← nextfree
    nextfree ← current
    start ← Make[current].pointer
    Make[current].pointer ← temp
```



```
IF previous IS NULL   //first node to be deleted
    temp ← nextfree
    nextfree ← current
    start ← Make[current].pointer
    Make[current].pointer ← temp
```

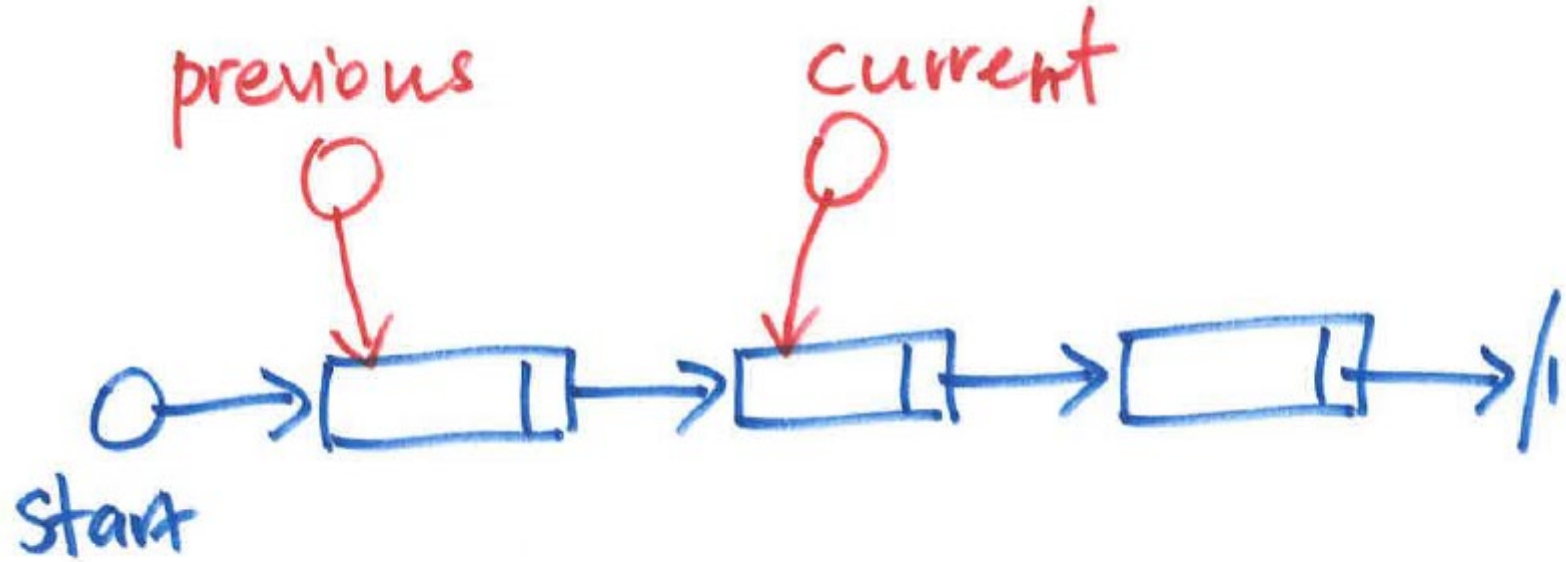


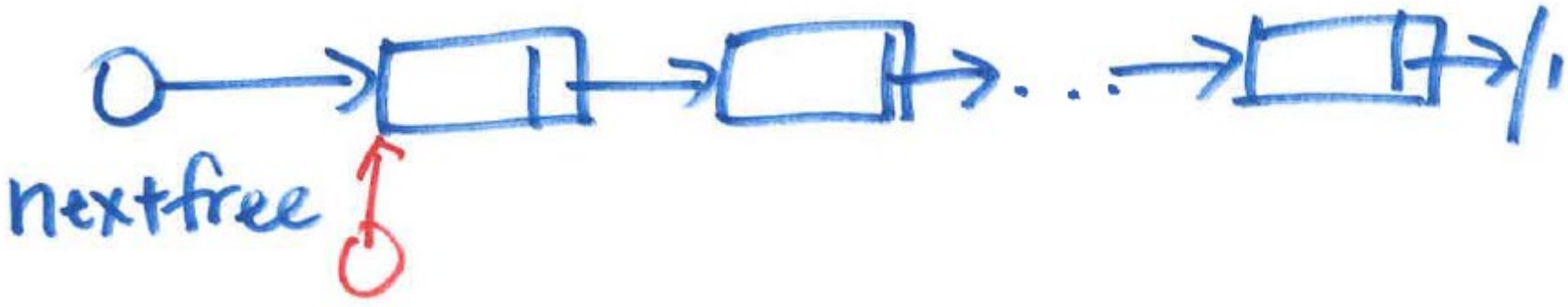
```
IF previous IS NULL   //first node to be deleted
    temp ← nextfree
    nextfree ← current
    start ← Make[current].pointer
    Make[current].pointer ← temp
```



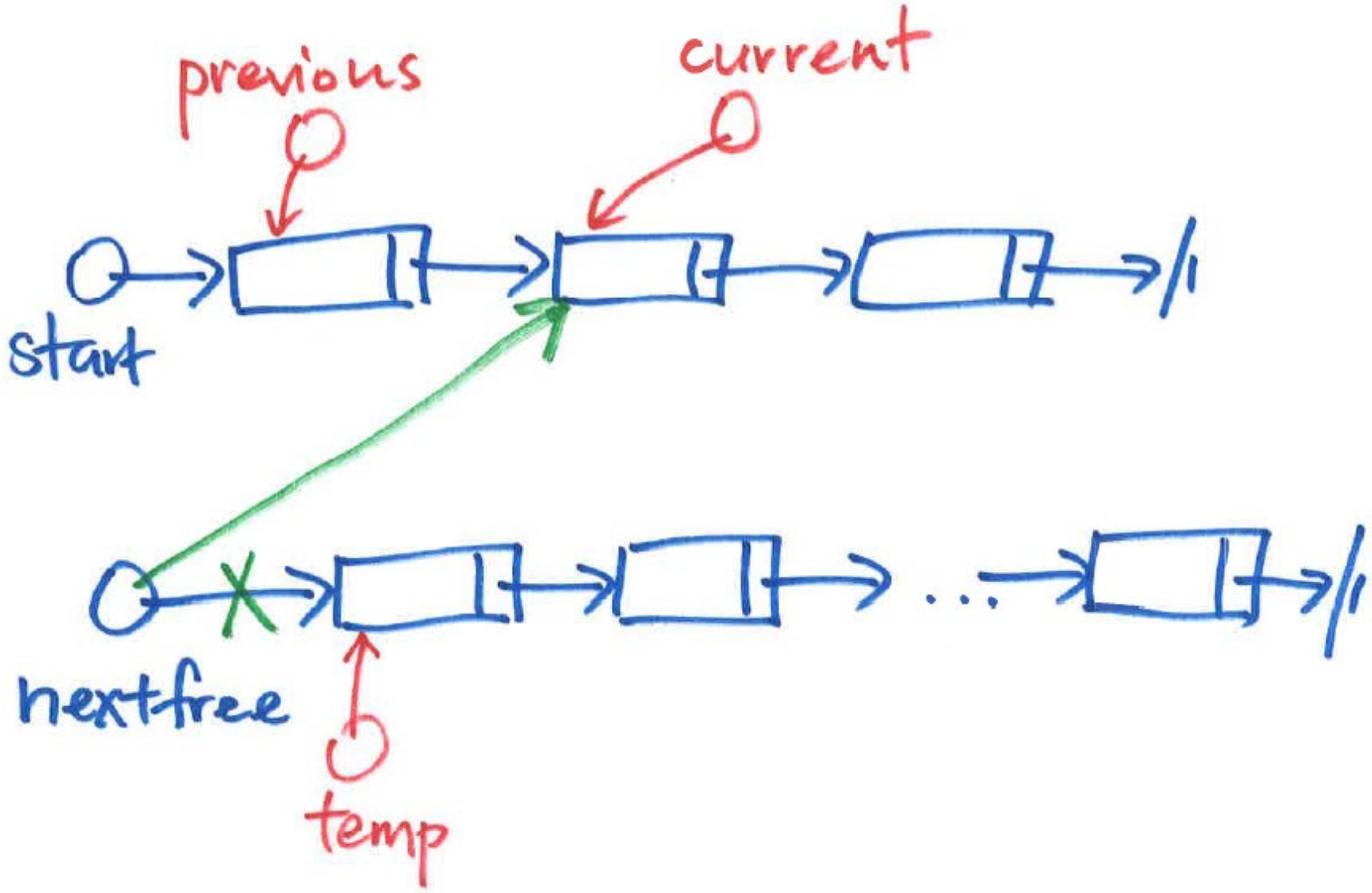
Delete non-1st node

```
ELSE    //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
```

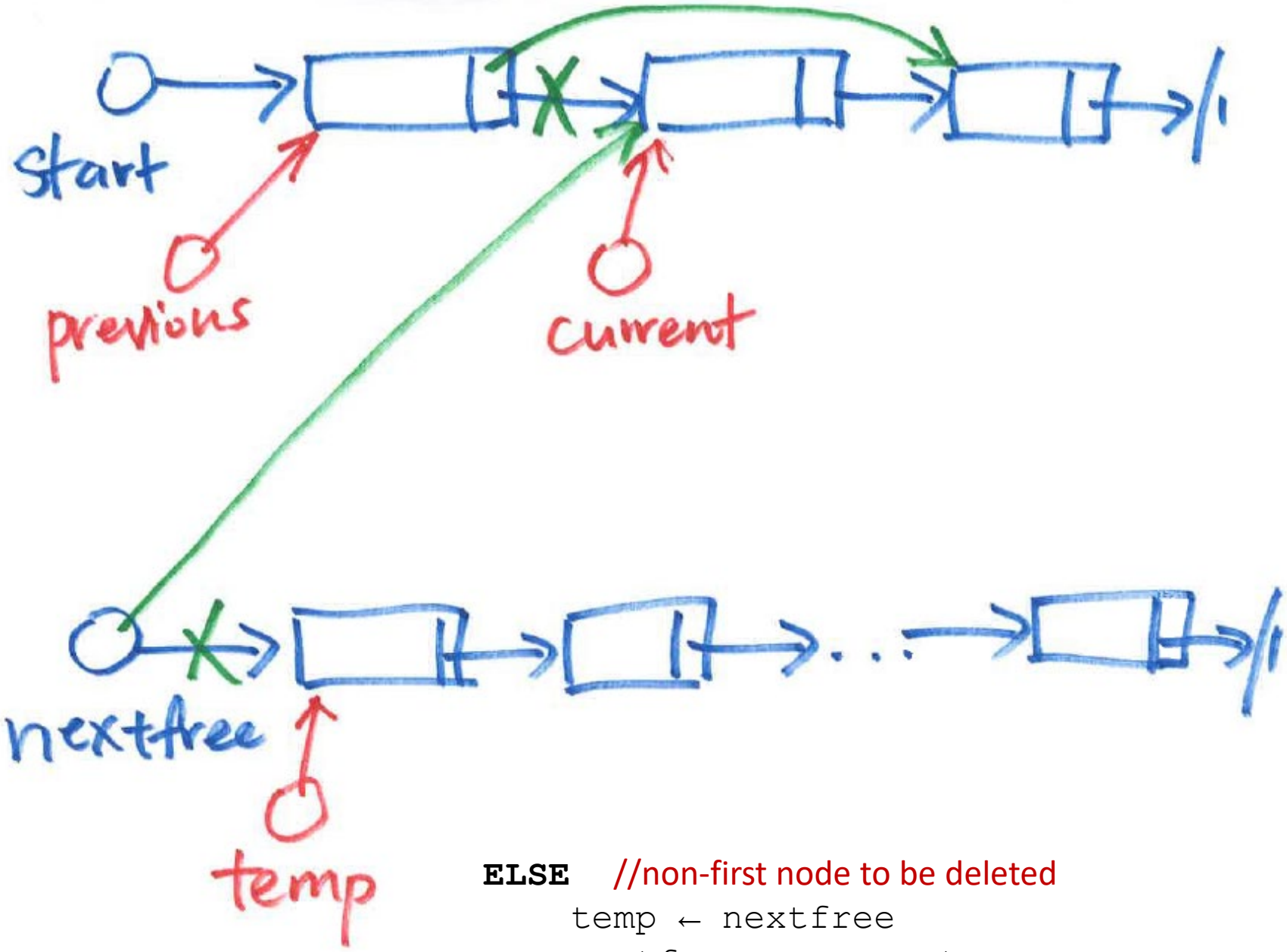




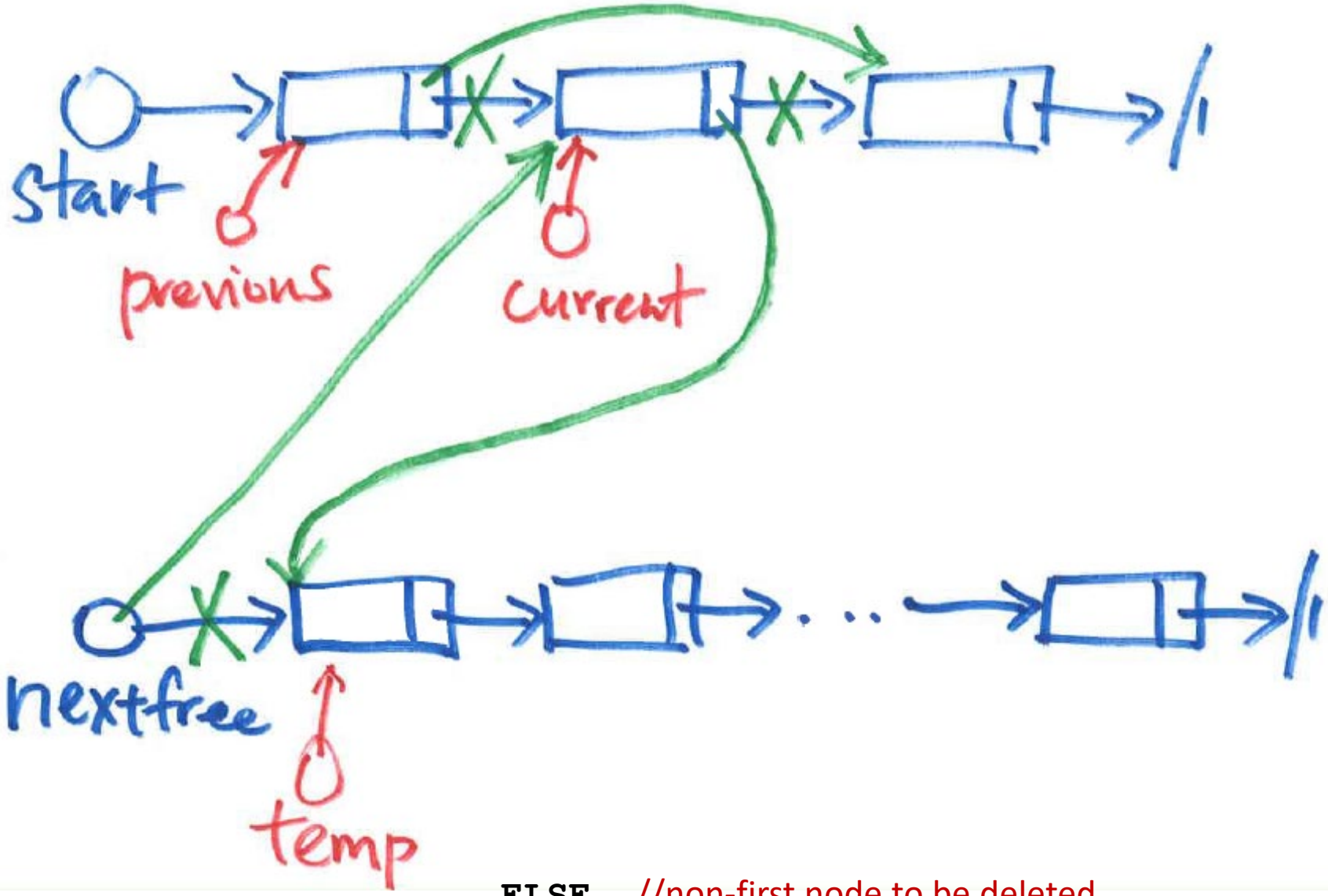

```
ELSE    //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
```




```
ELSE    //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
```




```
ELSE    //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
```




```
ELSE    //non-first node to be deleted
        temp ← nextfree
        nextfree ← current
        Make[previous].pointer ← Make[current].pointer
        Make[current].pointer ← temp
```

Linked List

Insert and Delete